

Reviewer Pack — Free Probability Tensor Entanglement and BP_ReadTwice Lower ...

Entry 2338b7d594c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 17:19:28 UTC

Free Probability Tensor Entanglement and BP_ReadTwice Lower Bounds

Entry ID: 2338b7d594c4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 17:19:28 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Branching Program: read-once vs read-twice (BP)

Statement:

[‘For any deterministic polynomial-time algorithm solving a random instance of a BP with size n , the free probability tensor entanglement $\rho(P)$ of the BP P is upper bounded by $O(\log n)$, where $\rho(P) = \max_{\{1 \leq i \leq j \leq n\}} |T_i \otimes T_j|$, and T_i denotes the conditional distribution of the i -th bit.’, ‘For any random BP IP_2 with size n , the free probability tensor entanglement $\rho(IP_2)$ is lower bounded by $\Omega(\sqrt{n})$.’, ‘As a corollary, if there exists a polynomial-time algorithm with a read-twice BP of size $O(n^c)$ for some $c < 1.5$ that has free probability tensor entanglement $\rho(P) = o(\log n)$, then $P \neq NP$.’]

Rationale (proposer’s reasoning):

[‘Free probability theory provides a framework to study the non-commutative probability spaces, which can model quantum information. The tensor entanglement in this context measures the degree of correlation between different parts of the system.’, ‘The conjecture suggests that BP complexity and free probability tensor entanglement are fundamentally linked, providing a potential avenue for proving $P \neq NP$ by leveraging the non-classical nature of quantum information theory.’, ‘This connection has not been explored before in the context of BP complexity, making it an under-explored area with the potential to expose new structures.’]

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a947e08c2e5ec7b5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For BP instances, if the maximum free probability tensor entanglement $\rho(P)$ is $\leq O(\log n)$ with a mean across 30 seeds of ≤ 2.5 ; and for IP_2 instances, if the mean free probability tensor entanglement $\rho(IP_2)$ across 30 seeds is $\geq \Omega(n)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'free probability theory' AND 'BP read-once vs read-twice lower bounds' - 'tensor entanglement' AND 'branching program free probability' - 'IP_2 branching program' AND 'free probability tensor entanglement lower bound'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(n):
        bp = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        return bp

    def matrix_multiply(A, B):
        m, p = len(A), len(B[0])
        n = len(B)
        result = [[Fraction(0) for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    result[i][j] += A[i][k] * B[k][j]
        return result

    def free_probability_tensor_entanglement(bp, ip_2):
        n = len(bp)
        T_i = [[Fraction(0) for _ in range(n)] for _ in range(n)]
        T_j = [[Fraction(0) for _ in range(n)] for _ in range(n)]
```

```

# Calculate  $T_i$  and  $T_j$ 
for i in range(n):
    for j in range(n):
        if bp[i][j] == 1:
             $T_i[i][j]$  = Fraction(1)
             $T_j[j][i]$  = Fraction(1)

entanglement = 0
for i in range(n):
    for j in range(i, n):
        product = matrix_multiply( $T_i[i]$ ,  $T_j[j]$ )
        for k in range(n):
            for l in range(n):
                if abs(product[k][l]) > entanglement:
                    entanglement = abs(product[k][l])
return entanglement

def check_bp():
    n = random.randint(5, 40)
    bp = generate_bp(n)
    entanglement = free_probability_tensor_entanglement(bp, None)
    if entanglement > Fraction(math.log(n), 1):
        return {"metric_name": "entanglement", "metric_value": float(entanglement), "instances": 1}
    else:
        return {"metric_name": "entanglement", "metric_value": float(entanglement), "instances": 0}

def check_ip_2():
    n = random.randint(5, 40)
    ip_2 = generate_bp(n)
    entanglement = free_probability_tensor_entanglement(None, ip_2)
    if entanglement < Fraction(n, 1):
        return {"metric_name": "entanglement", "metric_value": float(entanglement), "instances": 1}
    else:
        return {"metric_name": "entanglement", "metric_value": float(entanglement), "instances": 0}

trial_result = check_bp()
if not trial_result["conjecture_holds"]:
    return trial_result

trial_result.update(check_ip_2())
return trial_result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(100, 999) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    entanglement_values = [r["metric_value"] for r in results if "entanglement" in r]
    conjecture_holds = all(r["conjecture_holds"] for r in results if "conjecture_holds" in r)

```

```

mean_entanglement = sum(entanglement_values) / len(entanglement_values)
std_dev = math.sqrt(sum((x - mean_entanglement) ** 2 for x in entanglement_values) / len(entan
support_fraction = len([r for r in results if "conjecture_holds" in r and r["conjecture_holds"]

if conjecture_holds:
    print(f"RESULT: SUPPORTED mean={mean_entanglement} std={std_dev} support_fraction={support
elif any("counterexample" in r for r in results):
    counterexample = next(r["counterexample"] for r in results if "counterexample" in r)
    first_failing_seed = next(r["seed"] for r in results if "conjecture_holds" not in r or not
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3a9cd4b.py", line 88, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3a9cd4b.py", line 75, in run_trial
    trial_result = check_bp()
                    ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3a9cd4b.py", line 60, in check_bp
    entanglement = free_probability_tensor_entanglement(bp, None)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3a9cd4b.py", line 50, in free_probabil
    product = matrix_multiply(T_i[i], T_j[j])
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3a9cd4b.py", line 26, in matrix_multip
    m, p = len(A), len(B[0])
               ^^^^^^^^^
TypeError: object of type 'Fraction' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, re-run the test with appropriate BP and IP₂ instances to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13301
2	propose	ollama_remote	glm4:latest	0	0	14985
3	preregistration	ollama_remote	glm4:latest	0	0	10547
4	novelty	ollama_remote	glm4:latest	0	0	8837
5	novelty	ollama_remote	glm4:latest	0	0	9785
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13890
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14475
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12075
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12870
10	judge	ollama_remote	glm4:latest	0	0	12569

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123332 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2338b7d594c4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2338b7d594c4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2338b7d594c4.tar.gz` (if generated)